06. 동기화 시스템 분석

개요

Reth의 동기화 시스템은 **Staged Sync** 아키텍처를 기반으로 구현되어 있습니다. 이 시스템은 블록체인 동기화 과정을 여러 개의 독립적인 단계(Stage)로 분할하여 각각을 순차적으로 실행하는 방식을 채택합니다. 각 단계는 특정한 책임을 가지며, 전체 파이프라인을 통해 체계적으로 블록체인 데이터를 동기화합니다.

핵심 구조체 및 트레이트

Pipeline 구조체

```
pub  struct Pipeline<N: ProviderNodeTypes> {

/// Provider factory

provider_factory: ProviderFactory<N>,

/// 실행 순서대로 구성된 모든 스테이지들

stages: Vec<BoxedStage<<ProviderFactory<N> as DatabaseProviderFactory>::Provider

/// 동기화할 최대 블록 번호

max_block: Option<BlockNumber>,

/// 정적 파일 생성기

static_file_producer: StaticFileProducer<ProviderFactory<N>>,

/// 파이프라인 이벤트 발송기

event_sender: EventSender<PipelineEvent>,
```

```
/// 파이프라인 진행 상황 추적
progress: PipelineProgress,
/// 체인 팁 동기화를 위한 송신기
tip_tx: Option<watch::Sender<B256>>,
/// 메트릭 이벤트 송신기
metrics_tx: Option<MetricEventsSender>,
}
```

Stage 트레이트

```
pub trait Stage<Provider>: Send + Sync {
/// 스테이지의 고유 ID 반환
fn id(&self) -> StageId;
/// 스테이지 실행 준비 상태 확인 (비동기)
fn poll_execute_ready(
&mut self,
_cx: &mut Context<'_>,
_input: ExecInput,
) -> Poll<Result<(), StageError>>;
/// 스테이지 실행 (순방향 동기화)
fn execute(&mut self, provider: &Provider, input: ExecInput) -> Result<ExecOut
/// 스테이지 되감기 (역방향 동기화)
fn unwind(&mut self, provider: &Provider, input: UnwindInput) -> Result<Unwind
```

```rust
/// 실행 후 커밋 훅
fn post_execute_commit(&mut self) -> Result<(), StageError>;
/// 되감기 후 커밋 훅
fn post_unwind_commit(&mut self) -> Result<(), StageError>;
}
```

실행 입력/출력 구조체

```rust
/// 스테이지 실행 입력
#[derive(Debug, Default, PartialEq, Eq, Clone, Copy)]
pub struct ExecInput {
/// 목표 블록 번호
pub target: Option<BlockNumber>,
/// 이전 실행의 체크포인트
pub checkpoint: Option<StageCheckpoint>,
}

/// 스테이지 실행 출력
#[derive(Debug, PartialEq, Eq, Clone)]
pub struct ExecOutput {
/// 도달한 체크포인트
pub checkpoint: StageCheckpoint,
```

```
/// 완료 여부
pub done: bool,
}
```

주요 기능

1. 파이프라인 실행 흐름

run_loop() 메서드

파이프라인의 핵심 실행 로직으로, 모든 스테이지를 순차적으로 실행합니다:

```
pub async fn run_loop(&mut self) -> Result<ControlFlow, PipelineError> {
// 정적 파일로 데이터 이동
self.move_to_static_files()?;

let mut previous_stage = None;
for stage_index in 0..self.stages.len() {
let stage = &self.stages[stage_index];
let stage_id = stage.id();

trace!(target: "sync::pipeline", stage = %stage_id, "Executing stage");
let next = self.execute_stage_to_completion(previous_stage, stage_index).await?

match next {
```

```
ControlFlow::NoProgress { block_number } => {
if  let Some(block_number) = block_number {
self.progress.update(block_number);
}
}
ControlFlow::Continue { block_number } => self.progress.update(block_number),
ControlFlow::Unwind { target, bad_block } => {
self.unwind(target, Some(bad_block.block.number))?;
return Ok(ControlFlow::Unwind { target, bad_block })
}
}


previous_stage = Some(
self.provider_factory
.provider()?
.get_stage_checkpoint(stage_id)?
.unwrap_or_default()
.block_number,
);
}


Ok(self.progress.next_ctrl())
```

```
}
```

execute_stage_to_completion() 메서드

개별 스테이지를 완료까지 실행하는 로직:

```
async fn execute_stage_to_completion(
&mut self,
previous_stage: Option<BlockNumber>,
stage_index: usize,
) -> Result<ControlFlow, PipelineError> {
let stage_id = self.stage(stage_index).id();
let mut made_progress = false;
let target = self.max_block.or(previous_stage);

loop {
let prev_checkpoint = self.provider_factory.get_stage_checkpoint(stage_id)?;
let exec_input = ExecInput { target, checkpoint: prev_checkpoint };

// 스테이지 실행 준비 대기
if let Err(err) = self.stage(stage_index).execute_ready(exec_input).await {
// 오류 처리 로직
}

// 스테이지 실행
```

```
match  self.stage(stage_index).execute(&provider_rw, exec_input) {
Ok(out @ ExecOutput { checkpoint, done }) => {
// 체크포인트 저장 및 커밋
provider_rw.save_stage_checkpoint(stage_id, checkpoint)?;
UnifiedStorageWriter::commit(provider_rw)?;
self.stage(stage_index).post_execute_commit()?;


if  done {
return Ok(if  made_progress {
ControlFlow::Continue { block_number }
} else {
ControlFlow::NoProgress { block_number: Some(block_number) }
})
}
}
Err(err) => {
// 오류 처리 및 되감기 로직
}
}
}
}
```

2. 되감기 (Unwind) 메커니즘

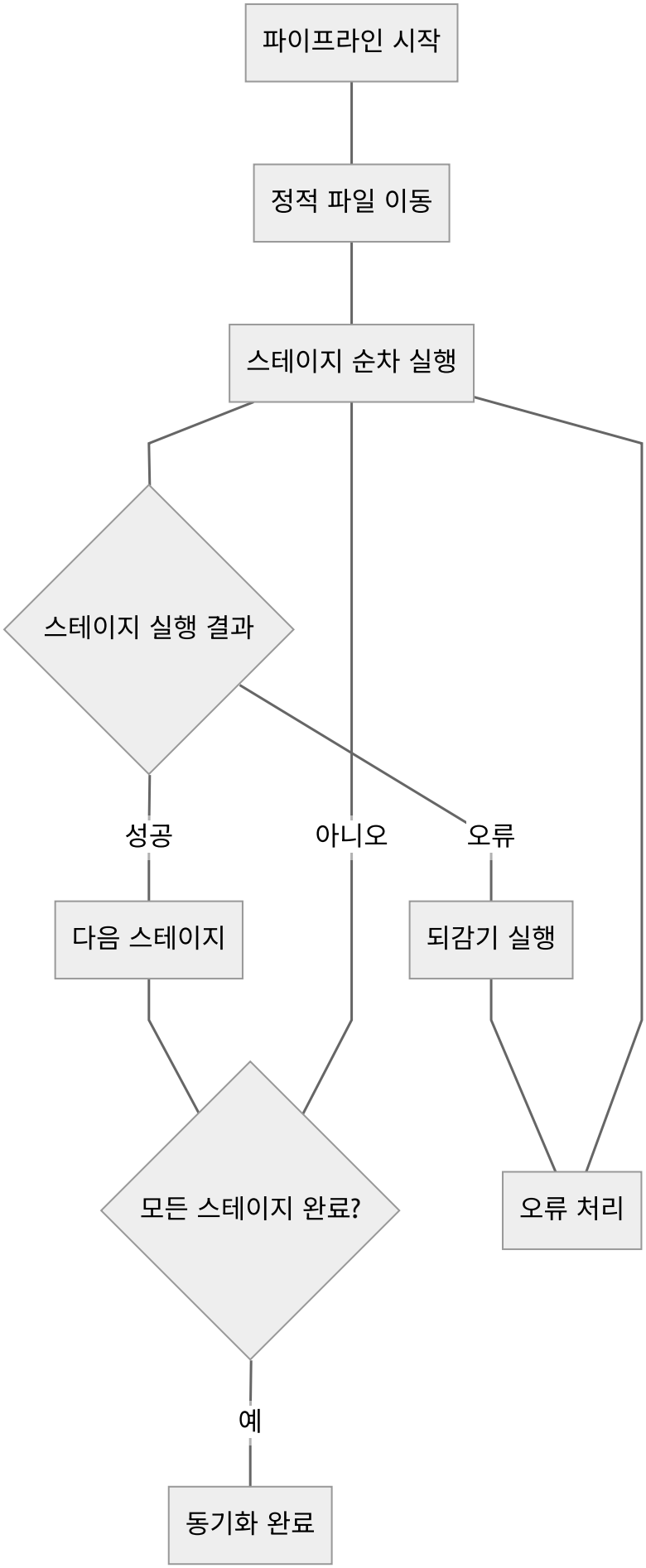
잘못된 블록이나 검증 오류 발생 시 스테이지들을 역순으로 되감는 기능:

```
pub fn unwind(
&mut self,
to: BlockNumber,
bad_block: Option<BlockNumber>,
) -> Result<(), PipelineError> {
// 스테이지들을 역순으로 되감기
let unwind_pipeline = self.stages.iter_mut().rev();
let mut provider_rw = self.provider_factory.database_provider_rw()?;


for stage in unwind_pipeline {
let stage_id = stage.id();
let mut checkpoint = provider_rw.get_stage_checkpoint(stage_id)?.unwrap_or_def
if checkpoint.block_number < to {
continue; // 이미 목표보다 낮은 경우 스킵
}


while checkpoint.block_number > to {
let input = UnwindInput { checkpoint, unwind_to: to, bad_block };
let output = stage.unwind(&provider_rw, input)?;
checkpoint = output.checkpoint;
provider_rw.save_stage_checkpoint(stage_id, checkpoint)?;
```

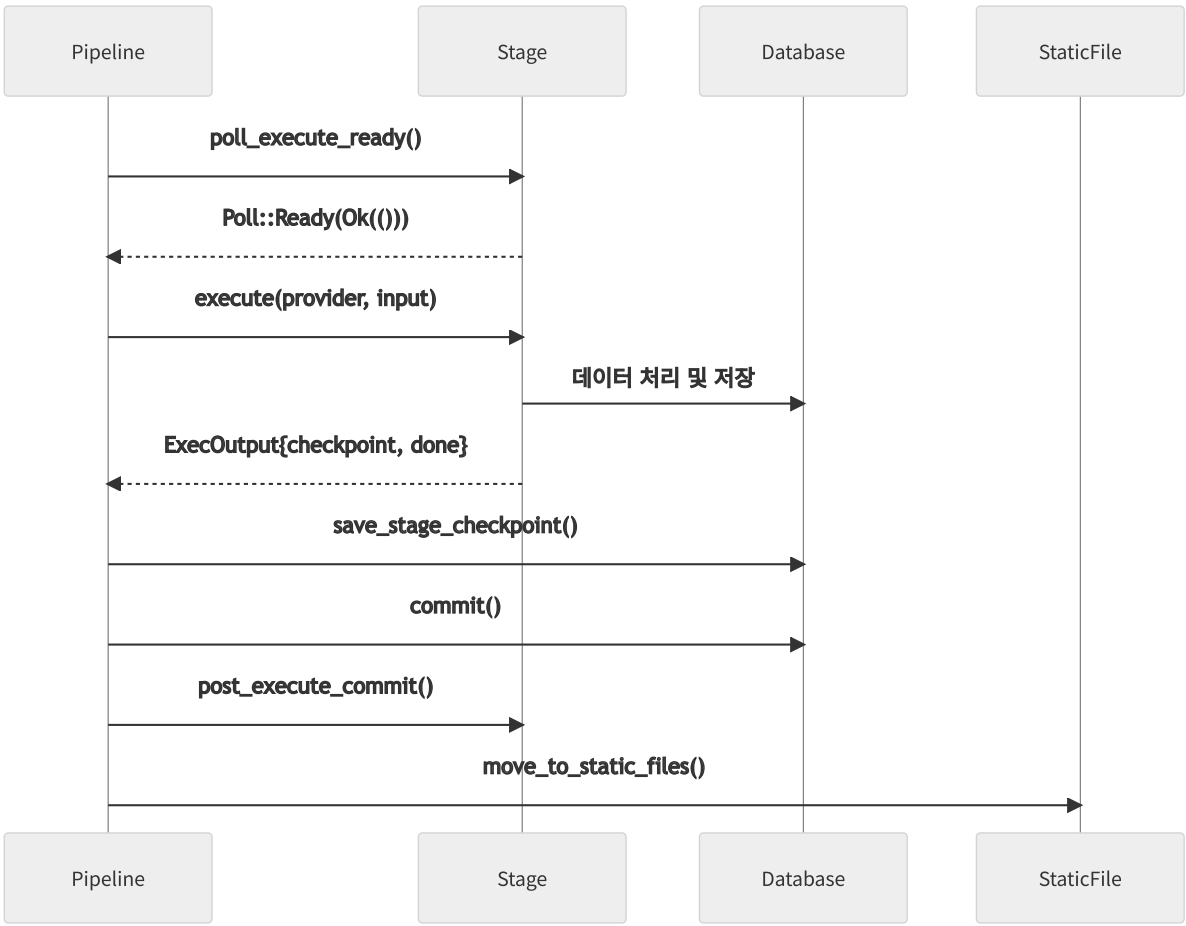
```
UnifiedStorageWriter::commit_unwind(provider_rw)?;

stage.post_unwind_commit()?;

provider_rw = self.provider_factory.database_provider_rw()?;

}

}



Ok(())

}
```

프로세스 플로우

1. 전체 동기화 과정



2. 개별 스테이지 실행 과정


Pipeline
Stage
Database
StaticFile
poll_execute_ready()
Poll::Ready(Ok(()))
execute(provider, input)
데이터 처리 및 저장
ExecOutput{checkpoint, done}
save_stage_checkpoint()
commit()
post_execute_commit()
move_to_static_files()
Pipeline
Stage
Database
StaticFile

3. 되감기 과정

각 동기화 단계별 처리 과정

1. 온라인 스테이지 (OnlineStages)

네트워크 접근이 필요한 스테이지들:

EraStage

- **역할**: ERA1 파일에서 히스토리컬 데이터 가져오기
- **처리**: 압축된 히스토리컬 블록 데이터 처리

HeaderStage

- **역할**: 블록 헤더 다운로드 및 검증
- **처리**: P2P 네트워크에서 헤더 체인 동기화
- **핵심 로직**:

```
impl<Provider, P, D> Stage<Provider> for HeaderStage<P, D>
where
Provider: DBProvider<Tx: DbTxMut> + StaticFileProviderFactory,
{
fn execute(&mut self, provider: &Provider, input: ExecInput) -> Result<ExecOut
// 헤더 다운로드 및 검증 로직
// 연속성 검사 및 난이도 검증
// 정적 파일에 헤더 저장
}
}
```

BodyStage

- **역할**: 블록 바디 (트랜잭션, 영수증) 다운로드
- **처리**: 헤더에 대응하는 블록 바디 데이터 수집

2. 실행 스테이지 (ExecutionStages)

SenderRecoveryStage

- **역할**: 트랜잭션 서명에서 발신자 주소 복구

- **병렬 처리**: Rayon을 사용한 멀티스레드 서명 복구
- **핵심 최적화**:

```
fn setup_range_recovery<Provider>(
provider: &Provider,
// ... 매개변수들
) {
// 전역 rayon 풀을 사용한 병렬 처리
// 청크 단위로 트랜잭션 배치 처리
// 메모리 효율적인 스트리밍 처리
}
```

ExecutionStage

- **역할**: EVM을 통한 트랜잭션 실행
- **처리**: 상태 변경, 영수증 생성, 가스 소모 계산
- **최적화**: 병렬 실행 및 상태 캐싱

3. 해싱 스테이지 (HashingStages)

AccountHashingStage

- **역할**: 계정 상태의 해시 계산
- **처리**: 변경된 계정들의 해시 업데이트

StorageHashingStage

- **역할**: 스토리지 슬롯의 해시 계산

- **처리**: 컨트랙트 스토리지 변경사항 해싱

MerkleStage

- **역할**: 머클 트리 구성 및 상태 루트 계산
- **최적화**: 증분 업데이트 및 재구성 임계값 설정

4. 인덱싱 스테이지 (HistoryIndexingStages)

TransactionLookupStage

- **역할**: 트랜잭션 해시 → 블록 번호 인덱스 생성
- **최적화**: ETL (Extract, Transform, Load) 패턴 사용

IndexAccountHistoryStage / IndexStorageHistoryStage

- **역할**: 계정/스토리지 히스토리 인덱스 생성
- **처리**: 히스토리컬 상태 조회를 위한 인덱스 구축

5. 정리 스테이지

PruneStage

- **역할**: 불필요한 데이터 정리
- **처리**: 설정된 보존 정책에 따른 데이터 삭제

FinishStage

- **역할**: 동기화 완료 처리
- **처리**: 최종 상태 검증 및 메트릭 업데이트

병렬 처리 및 성능 최적화 방식

1. 비동기 처리 아키텍처

스테이지 준비 상태 관리

```rust
fn poll_execute_ready(
&mut self,
_cx: &mut Context<'_>,
_input: ExecInput,
) -> Poll<Result<(), StageError>> {
// 비동기 리소스 준비
// 다운로더 상태 확인
// 내부 버퍼 관리
Poll::Ready(Ok(()))
}
```

논블로킹 실행 준비

- `execute_ready()` 메서드를 통한 비동기 준비
- 다운로더와의 비동기 통신
- 내부 버퍼 및 리소스 사전 할당

2. 메모리 최적화

ETL (Extract, Transform, Load) 패턴

```rust
// TransactionLookupStage에서의 ETL 사용 예시
pub struct TransactionLookupStage {
```

```
etl_config: EtlConfig,
// 대용량 데이터를 디스크 기반으로 처리
}
```

청크 기반 처리

- 대용량 데이터를 작은 청크로 분할
- 메모리 사용량 제한 및 예측 가능한 성능
- 점진적 커밋을 통한 안정성 확보

3. 병렬 처리 전략

Rayon 기반 데이터 병렬성

```
// SenderRecoveryStage에서의 병렬 처리
fn setup_range_recovery<Provider>(
provider: &Provider,
// ...
) {
// 전역 rayon 풀 사용
// CPU 코어 수에 맞춘 병렬 처리
// 작업 도난(work-stealing) 스케줄링
}
```

파이프라인 병렬성

- 스테이지 간 의존성 최소화
- 독립적인 스테이지들의 동시 실행 가능성
- 체크포인트 기반 진행 상황 추적

4. I/O 최적화

정적 파일 시스템 활용

```
pub fn move_to_static_files(&self) -> RethResult<()> {
// 데이터베이스에서 정적 파일로 이동
let lowest_static_file_height =
self.static_file_producer.lock().copy_to_static_files()?.min_block_num();
// 이동된 데이터 정리
if let Some(prune_tip) = lowest_static_file_height {
let mut pruner = PrunerBuilder::new(Default::default())
.delete_limit(usize::MAX)
.build_with_provider_factory(self.provider_factory.clone());
pruner.run(prune_tip)?;
}
Ok(())
}
```

배치 커밋 전략

- 트랜잭션 크기 최적화
- 커밋 임계값 설정을 통한 성능 조절

- 메모리와 디스크 I/O 균형

5. 오류 복구 최적화

체크포인트 기반 재시작

- 각 스테이지별 독립적인 체크포인트
- 오류 발생 시 최소한의 되감기
- 점진적 진행 상황 저장

적응적 되감기 깊이

```
// DetachedHead 오류 처리에서의 적응적 되감기
let  unwind_to = local_head
.block
.number
.saturating_sub(
BEACON_CONSENSUS_REORG_UNWIND_DEPTH.saturating_mul(self.detached_head_attempts),
)
.max(1);
```

설정 및 커스터마이징

1. 스테이지 설정

StageConfig 구조체

```rust
pub struct StageConfig {
pub sender_recovery: SenderRecoveryConfig,
pub execution: ExecutionConfig,
pub account_hashing: HashingConfig,
pub storage_hashing: HashingConfig,
pub merkle: MerkleConfig,
pub transaction_lookup: TransactionLookupConfig,
pub index_account_history: IndexHistoryConfig,
pub index_storage_history: IndexHistoryConfig,
pub prune: PruneConfig,
pub etl: EtlConfig,
}
```

개별 스테이지 설정 예시

```rust
pub struct ExecutionConfig {
/// 최대 블록 배치 크기
pub max_blocks: Option<u64>,
/// 외부 정리 임계값
pub external_clean_threshold: u64,
}

pub struct MerkleConfig {
```

```
/// 재구성 임계값
pub rebuild_threshold: u64,
/// 증분 업데이트 임계값
pub incremental_threshold: u64,
}
```

2. 파이프라인 빌더

PipelineBuilder 사용법

```
let pipeline = Pipeline::<MockNodeTypesWithDB>::builder()
.add_stages(DefaultStages::new(
provider,
tip,
consensus,
header_downloader,
body_downloader,
evm_config,
stages_config,
prune_modes,
era_import_source,
))
.with_max_block(target_block)
```

```
.build(provider_factory, static_file_producer);
```

커스텀 스테이지 추가

```
let  pipeline = Pipeline::builder()
.add_stage(CustomStage::new())
.add_set(DefaultStages::new(/* ... */))
.build(provider_factory, static_file_producer);
```

3. 성능 튜닝 매개변수

메모리 관련 설정

- ETL 버퍼 크기 조정
- 커밋 임계값 설정
- 청크 크기 최적화

병렬성 설정

- Rayon 스레드 풀 크기
- 비동기 작업 동시성 수준
- I/O 배치 크기

관련 컴포넌트

1. 의존성 관계

- **ProviderFactory**: 데이터베이스 접근 제공

- **StaticFileProducer**: 정적 파일 관리
- **Consensus**: 블록 검증 로직
- **EVM**: 트랜잭션 실행 엔진
- **NetworkHandle**: P2P 통신 (온라인 스테이지용)

2. 상호작용 방식

이벤트 시스템

```
pub  enum PipelineEvent {
Prepare { stage_id: StageId, /* ... */ },
Run { stage_id: StageId, /* ... */ },
Ran { stage_id: StageId, result: ExecOutput },
Unwind { stage_id: StageId, input: UnwindInput },
Unwound { stage_id: StageId, result: UnwindOutput },
Error { stage_id: StageId },
Skipped { stage_id: StageId },
}
```

메트릭 수집

- 각 스테이지별 실행 시간 측정
- 처리된 블록 수 및 진행률 추적
- 오류 발생률 및 되감기 빈도 모니터링

Reth의 Staged Sync 아키텍처는 복잡한 블록체인 동기화 과정을 체계적이고 효율적으로 관리할 수 있게 해주는 강력한 시스템입니다. 각 스테이지의 독립성과 병렬 처리 최적화를 통해 높은 성능과 안

정성을 제공합니다.